Unit 5. MySQL SubQueries, Join, Indexing and Views

MySQL Subquery:

A query inside another query is called as Subquery. It is also called as nested query embedded with SELECT, INSERT, UPDATE or DELETE statement along with the various operators.

A subquery is known as the inner query, and the query that contains subquery is known as the outer query.

The inner query executed first gives the result to the outer query, and then the main/outer query will be performed. MySQL allows us to use subquery anywhere, but it must be closed within parenthesis.

The following are the rules to use subqueries:

- Subqueries should always use in parentheses.

-If the main query does not have multiple columns for subquery, then a subquery can have only one column in the SELECT command.

-We can use various comparison operators with the subquery, such as >, <, =, IN, ANY, SOME, and ALL.

-A multiple-row operator is very useful when the subquery returns more than one row.

-We cannot use the ORDER BY clause in a subquery, although it can be used inside the main query.

-If we use a subquery in a set function, it cannot be immediately enclosed in a set function.

Subquery Syntax:

```
SELECT column_list (s) FROM  table_name  WHERE  column_name OPERATOR
  (SELECT column_list (s)  FROM table_name [WHERE]) ;
```

MySQL Subquery Example

Let us understand it with the help of an example. Suppose we have a table named "employees"

```
CREATE TABLE employees(id INT ,name varchar(20) , age INT , city VARCHAR(15) ,income INT);
```

Statement to insert values in table employees

```
INSERT INTO employees VALUES(101, "Mahesh",35,”Pune”,29000), (102,”Anant”,33,”Kolhapur”,23000),
(103,”Pradeep”,39,”Kolhapur”,40000),(104,”Chetan”,31,”Solapur”,21000),
(105,”Ganesh”,31,”Sangli”,20000);
```

```
mysql> select * from employees;
+------+---------+------+----------+--------+
| id   | name    | age  | city     | income |
+------+---------+------+----------+--------+
|  101 | Mahesh  |   35 | Pune     |  29000 |
|  102 | Anant   |   33 | Kolhapur |  23000 |
|  103 | Pradeep |   39 | Kolhapur |  40000 |
|  104 | Chetan  |   31 | Solapur  |  21000 |
|  105 | Ganesh  |   31 | Sangli   |  20000 |
+------+---------+------+----------+--------+
5 rows in set (0.07 sec)
```

Now let's take a look of subqueries.

Following is a simple SQL statement that returns the employee detail whose id matches in a subquery:

```
SELECT name, city, income FROM employees
 WHERE id IN (SELECT id FROM employees);
```

name	city	income
Mahesh	Pune	29000
Anant	Kolhapur	23000
Pradeep	Kolhapur	40000
Chetan	Solapur	21000
Ganesh	Sangli	20000

MySQL Subquery with Comparison Operator:

A comparison operator is an operator used to compare values and returns the result, either true or false. The following comparison operators are used in MySQL <, >, =, <>, <=>, etc.

We can use the subquery before or after the comparison operators that return a single value. The returned value can be the arithmetic expression or a column function. After that, SQL compares the subquery results with the value on the other side of the comparison operator.

Following is a simple SQL statement that returns the employee detail whose income is more than 25000 with the help of subquery:

```
SELECT * FROM employees
WHERE id IN
(SELECT id  FROM employees WHERE income > 25000);
```

id	name	age	city	income
101	Mahesh	35	Pune	29000
103	Pradeep	39	Kolhapur	40000

This query first executes the subquery that returns the employee id whose income > 25000. Second, the main query will return the employees all details whose employee id are in the result set returned by the subquery.

Let us see an example of another comparison operator, such as equality (=) to find employee details with maximum income using a subquery.

```
SELECT name, city, income FROM employees
WHERE income = (SELECT MAX(income) FROM employees);
```

name	city	income
Pradeep	Kolhapur	40000

MySQL Subquery with IN or NOT-IN Operator

If the subquery produces more than one value, we need to use the IN or NOT IN operator with the WHERE clause.

Let's take an example, but before that we have to create two tables named "Customer" and "Salesman" .

Table Name: Customer

```
CREATE TABLE Customer(cid INT ,cname varchar(20) , city VARCHAR(15));
```

Table Name: Salesman

```
CREATE TABLE Salesman(sid INT ,sname varchar(20) , city VARCHAR(15));
```

Now let's insert data in these tables as

Table: Customer

```
INSERT INTO Customer
VALUES(2001, "Harsh", "Baroda"),
(2002 ,"Gita", "Pune"),
(2003 ,"Lalit ","Mumbai");
```

```
mysql> select * from Customer;
+------+--------+--------+
| cid  | cname  | city   |
+------+--------+--------+
| 2001 | Harsh  | Baroda |
| 2002 | Gita   | Pune   |
| 2003 | Lalit  | Mumbai |
+------+--------+--------+
```

Table : Salesman

```
INSERT INTO Salesman
VALUES(1001, "Prashnat ","Mumbai"),
(1002, "Rajesh", "Surat"),
 (1007 ,"Nitin" ,"Delhi"), (1003 ,"Amit" ,"Pune");
```

```
mysql> select * from Salesman;
+------+-----------+--------+
| sid  | sname     | city   |
+------+-----------+--------+
| 1001 | Prashnat  | Mumbai |
| 1002 | Rajesh    | Surat  |
| 1007 | Nitin     | Delhi  |
| 1003 | Amit      | Pune   |
+------+-----------+--------+
```

The following subquery with NOT IN operator returns the Customer detail who does not belong to cities where salesman lives(i.e. A customer who do not live in the city where salesman lives):

```
select * from Customer where city NOT IN
(select city from Salesman);
```

```
+------+-------+--------+
| cid  | cname | city   |
+------+-------+--------+
| 2001 | Harsh | Baroda |
+------+-------+--------+
```

The following subquery with IN operator returns the Customer detail who belong to cities of where salesman live (i.e. A customer who lives in the city where salesman lives):

```
select * from Customer where city IN
(select city from Salesman);
```

```
+------+--------+--------+
| cid  | cname  | city   |
+------+--------+--------+
| 2002 | Gita   | Pune   |
| 2003 | Lalit  | Mumbai |
+------+--------+--------+
```

MySQL Subquery in the FROM Clause

If we use a subquery in the FROM clause, MySQL will return the output from a subquery is used as a temporary table. We called this table as a derived table, inline views, or materialized subquery.

Consider the above table named Customer

```
mysql> select * from Customer;
+------+--------+--------+
| cid  | cname  | city   |
+------+--------+--------+
| 2001 | Harsh  | Baroda |
| 2002 | Gita   | Pune   |
| 2003 | Lalit  | Mumbai |
+------+--------+--------+
```

The following subquery returns all columns from table customer.

```
select cname FROM
(select * from customer) as i;
```

cname
Harsh
Gita
Lalit

MySQL Correlated Subqueries

A correlated subquery in MySQL is a subquery that depends on the outer query.

It uses the data from the outer query or contains a reference to a parent query that also appears in the outer query. MySQL evaluates it once from each row in the outer query.

Consider the table named employees as

```
mysql> select * from employees;
```

id	name	age	city	income
101	Mahesh	35	Pune	29000
102	Anant	33	Kolhapur	23000
103	Pradeep	39	Kolhapur	40000
104	Chetan	31	Solapur	21000
105	Ganesh	31	Sangli	20000
106	Nitin	30	Sangli	10000

```
SELECT name, city, income FROM employees em
WHERE
income > (SELECT AVG(income) FROM employees WHERE city = em.city);
```

name	city	income
Pradeep	Kolhapur	40000
Ganesh	Sangli	20000

MySQL Subqueries with EXISTS or NOT EXISTS:

The EXISTS operator is a Boolean operator that returns either true or false result. It is used with a subquery and checks the existence of data in a subquery. If a subquery returns any record at all, this operator returns true. Otherwise, it will return false.

The NOT EXISTS operator used for negation that gives true value when the subquery does not return any row. Otherwise, it returns false. Both EXISTS and NOT EXISTS used with correlated subqueries. The following example illustrates it more clearly. Suppose we have a table customer and order that contains the data as follows:

MySQL 8.0 Command Line Client

```
mysql> SELECT * FROM customer;
```

cust_id	name	occupation	age
101	Peter	Engineer	32
102	Joseph	Developer	30
103	John	Leader	28
104	Stephen	Scientist	45
105	Suzi	Carpenter	26
106	Bob	Actor	25
107	NULL	NULL	NULL

7 rows in set (0.00 sec)

```
mysql> SELECT * FROM Orders;
```

order_id	cust_id	prod_name	order_date
1	101	Laptop	2020-01-10
2	103	Desktop	2020-02-12
3	106	Iphone	2020-02-15
4	104	Mobile	2020-03-05
5	102	TV	2020-03-20

The below SQL statements uses EXISTS operator to find the name, occupation, and age of the customer who has placed at least one order.

```
SELECT name, occupation, age FROM customer C
WHERE EXISTS
(SELECT * FROM Orders O WHERE C.cust_id = O.cust_id);
```

name	occupation	age
Peter	Engineer	32
Joseph	Developer	30
John	Leader	28
Stephen	Scientist	45
Bob	Actor	25

This statement uses NOT EXISTS operator that returns the customer details who have not placed an order.

```
SELECT name, occupation, age FROM customer C
WHERE NOT EXISTS
(SELECT * FROM Orders O WHERE C.cust_id = O.cust_id);
```

name	occupation	age
Suzi	Carpenter	26
NULL	NULL	NULL

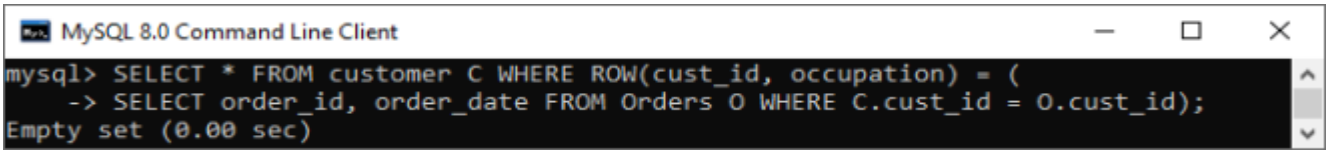
MySQL ROW Subqueries:

It is a subquery that returns a single row where we can get more than one column values. We can use the following operators for comparing row subqueries =, >, <, >=, <=, <>, !=, <=>.

Let us see the following example which is applied on above tables Customer and Order:

```
SELECT * FROM customer C WHERE ROW(cust_id, occupation) =
(SELECT order_id, order_date FROM Orders O WHERE C.cust_id = O.cust_id);
```

If given row has cust_id, occupation values equal to the order_id, order_date values of any rows in the first table, the WHERE expression is TRUE, and each query returns those first table rows. Otherwise, the expression is FALSE, and the query produces an empty set, which can be shown in the below image:



MySQL Subqueries with ALL, ANY, and SOME:

We can use a subquery which is followed by the keyword ALL, ANY, or SOME after a comparison operator. The following are the syntax to use subqueries with ALL, ANY, or SOME:

```
operand comparison_operator ANY (subquery)
operand comparison_operator ALL (subquery)
operand comparison  operator SOME (subquery)
```

The ALL keyword compares values with the value returned by a subquery. Therefore, it returns TRUE if the comparison is TRUE for ALL of the values returned by a subquery.

The ANY keyword returns TRUE if the comparison is TRUE for ANY of the values returned by a subquery.

The ANY and SOME keywords are the same because they are the alias of each other. The following example explains it more clearly:

Example of ANY/SOME

```
SELECT cust_id, name FROM customer WHERE
cust_id > ANY (SELECT cust_id FROM Orders);
```

cust_id	name
102	Joseph
103	John
104	Stephen
105	Suzi
106	Bob
107	NULL

Example of ALL

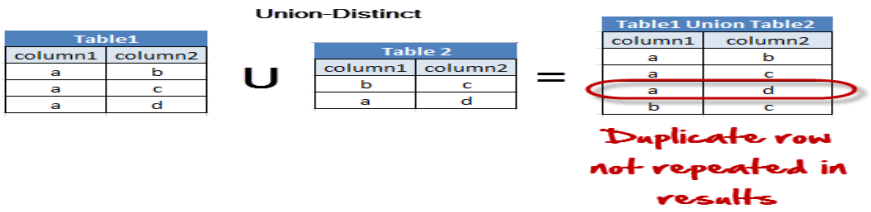
```
SELECT cust_id, name FROM customer WHERE
cust_id > ALL (SELECT cust_id FROM Orders);
```

cust_id	name
107	NULL

Set Operations

The MySQL Set operation is used to combine the two or more SQL SELECT statements.
Types of Sets in MySQL:
1) UNION 2)MINUS 3) INTERSECT

1) MySQL Union:
The MySQL UNION operator is used to combine the result sets of 2 or more SELECT statements. It removes duplicate rows between the various SELECT statements.
Each SELECT statement within the UNION operator must have the same number of fields in the result sets with similar data types.
MySQL always uses the name of the column in the first SELECT statement will be the column names of the result set(output).
We can understand the Union operator with the following visual representation:



Union Syntax:

```
SELECT column_list FROM table1  UNION  SELECT column_list FROM table2;
```

MySQL Union Example:
Let us create two tables named class1 and class 2, and see how the Union operator works in MySQL.

Create table Class1

```
CREATE TABLE class1(roll INT ,name varchar(20) , subject VARCHAR(15),marks INT);
```

Create table Class2

```
CREATE TABLE class2(roll INT ,name varchar(20) , subject VARCHAR(15),marks INT);
```

Now let's insert some values in both tables

Value insertion in Class1

```
INSERT INTO Class1
VALUES(1, "Harsh", "Physics",73), (2 ,"Gita", "CS",79),
(3 ,"Lalit ","English",85), (4 ,"Rahul ","CS",85);
```

```
mysql> select * from Class1;
+------+--------+---------+-------+
| roll | name   | subject | marks |
+------+--------+---------+-------+
|    1 | Harsh  | Physics |    73 |
|    2 | Gita   | CS      |    79 |
|    3 | Lalit  | English |    85 |
|    4 | Rahul  | CS      |    85 |
+------+--------+---------+-------+
```

Value insertion in Class2

```
INSERT INTO Class2
VALUES(1, "Nikhil", "History",77),(2 ,"Gita", "CS",79),
(3 ,"Aniket ","Geography",80);
```

```
mysql> select * from Class2;
+------+--------+-----------+-------+
| roll | name   | subject   | marks |
+------+--------+-----------+-------+
|    1 | Nikhil | History   |    77 |
|    2 | Gita   | CS        |    79 |
|    3 | Aniket | Geography |    80 |
+------+--------+-----------+-------+
```

Now let's perform union operation on both tables i.e. class1 and class2

```
SELECT name,subject FROM Class1
UNION
SELECT name,subject FROM Class2;
```

```
+---------+-----------+
| name    | subject   |
+---------+-----------+
| Harsh   | Physics   |
| Gita    | CS        |
| Lalit   | English   |
| Rahul   | CS        |
| Nikhil  | History   |
| Aniket  | Geography |
+---------+-----------+
6 rows in set (0.05 sec)
```

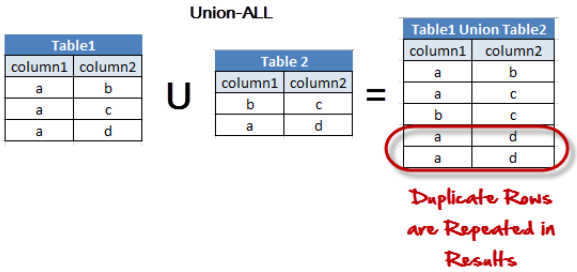
from class1

from class2

MySQL Union All

This operator returns all rows by combining two or more results from multiple SELECT queries into a single result set. It does not remove the duplicate rows from the result set.

We can understand it with the following pictorial representation:



Syntax:

```
SELECT column_list FROM table1  UNION ALL  SELECT column_list FROM table2;
```

Example of UNION ALL:

Consider above tables Class1 and Class2. Let's Perform UNION ALL Operation on both tables.

```
SELECT name,subject FROM Class1
UNION ALL
SELECT name,subject FROM Class2;
```

name	subject
Harsh	Physics
Gita	CS
Lalit	English
Rahul	CS
Nikhil	History
Gita	CS
Aniket	Geography

Duplicate Record

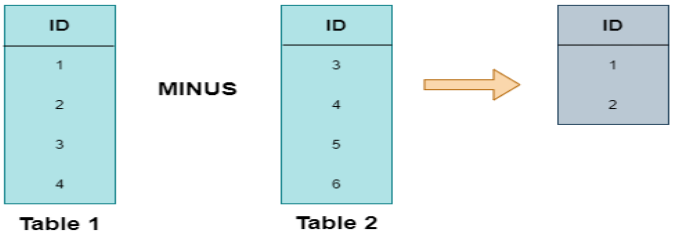
2) MySQL MINUS:

The MINUS operator returns the unique element from the first table/set, which is not found in the second table/set.

In other words, it will compare the results of two queries and produces the resultant row from the result set obtained by the first query and not found in the result set obtained by the second query

The following are the rules for a statement that uses the MINUS operator:

-The number and order of columns in all the SELECT statements must be the same.
-The data types of the corresponding columns in both SELECT statements must be the same or convertible.

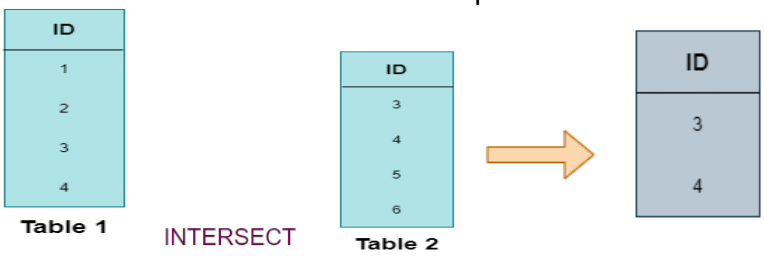


Since MySQL does not provide support for MINUS operator. However, we can use a LEFT JOIN clause to simulate this operator.

3) MySQL INTERSECT:

The INTERSECT operator returns the distinct (common) elements in two sets or common records from two or more tables. In other words, it compares the result obtained by two queries and produces unique rows, which are the result returned by both queries.

We can understand the INTERSECT operator with the following visual representation.



Since MySQL does not provide support for the INTERSECT operator. However, we can use the INNER JOIN and IN clause to emulate this operator.

MySQL Joins

Joins help retrieving data from two or more tables. The tables are mutually related using primary and foreign keys.

Types of joins:

1) INNER JOIN (simple join)
2) LEFT OUTER JOIN (LEFT JOIN)
3) RIGHT OUTER JOIN (RIGHT JOIN)
4) CROSS JOIN

Let's understand these Joins but before that we have to create two tables as,

Creating Table Movie

```
CREATE TABLE Movie(mid INT PRIMARY KEY, title VARCHAR(30), category VARCHAR(30));
```

Creating Table Viewer

```
CREATE TABLE Viewer(id INT, name VARCHAR(30),
mid int,FOREIGN KEY (mid)  REFERENCES Movie(mid));
```

Now let's insert some elements in these tables.

Table Movie:

```
INSERT INTO Movie
Values(1, " Real Steel(2012) ","Animation"),
(2," Lion King","Animation"),(3, "Madras Cafe ","Action"),
(4, " Avengers ","Action");
```

```
mysql> select * from Movie;
+-----+--------------------+-----------+
| mid | title              | category  |
+-----+--------------------+-----------+
|   1 |  Real Steel(2012)  | Animation |
|   2 |  Lion King         | Animation |
|   3 | Madras Cafe        | Action    |
|   4 |  Avengers          | Action    |
+-----+--------------------+-----------+
```

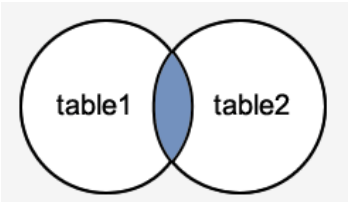
Table Viewer:

```
INSERT INTO Viewer Values(101, " Swapnil ",4),
(102, " Aakash",1),
(103, "Prashant ",3);
```

```
mysql> select * from viewer;
+------+-----------+------+
| id   | name      | mid  |
+------+-----------+------+
|  101 |  Swapnil  |    4 |
|  102 |  Aakash   |    1 |
|  103 | Prashant  |    3 |
+------+-----------+------+
```

Now let's perform different join operations on above two tables.

1) INNER JOIN (simple join)



The inner JOIN is used to return rows from both tables that satisfy the given condition.

Syntax of Inner Join

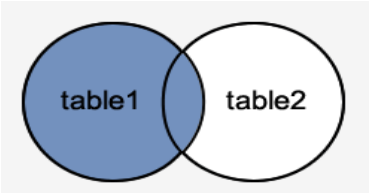
```
SELECT columns  FROM table1  INNER JOIN table2 ON table1.column = table2.column;
```

Example of Inner Join:

```
SELECT Viewer.id, Movie.mid ,title FROM Movie INNER JOIN Viewer
ON Movie.mid=Viewer.mid;
```

id	mid	title
101	4	Avengers
102	1	Real Steel(2012)
103	3	Madras Cafe

2) LEFT OUTER JOIN (LEFT JOIN)



The LEFT OUTER JOIN returns all rows from the left hand table specified in the ON condition and only those rows from the other table where the join condition is fulfilled.

Syntax of Left Outer Join

```
SELECT columns  FROM table1  LEFT JOIN table2 ON table1.column = table2.column;
```

Example of Left Outer Join

```
SELECT Movie.mid, title, Viewer.name FROM Movie LEFT JOIN
Viewer ON Movie.mid=Viewer.mid;
```

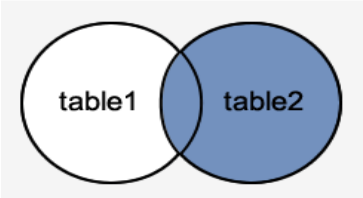
mid	title	name
1	Real Steel(2012)	Aakash
2	Lion King	NULL
3	Madras Cafe	Prashant
4	Avengers	Swapnil

4 rows in set (0.03 sec)

All records from movie

Viewer

3) RIGHT OUTER JOIN (RIGHT JOIN)



The MySQL Right Outer Join returns all rows from the Right-hand table specified in the ON condition and only those rows from the other table where the join condition is fulfilled.

Syntax of Right Outer Join

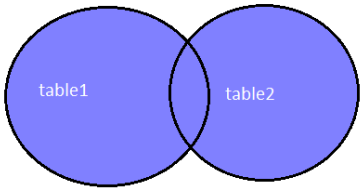
```
SELECT columns FROM table1 RIGHT JOIN table2 ON table1.column = table2.column;
```

Example of Right Outer Join

```
SELECT Movie.mid, title, Viewer.name FROM Movie RIGHT JOIN
Viewer ON Movie.mid=Viewer.mid;
```

mid	title	name
4	Avengers	Swapnil
1	Real Steel(2012)	Aakash
3	Madras Cafe	Prashant

4) CROSS JOIN



Cross JOIN is a simplest form of JOINs which matches each row from one database table to all rows of another.

In other words it gives us combinations of each row of first table with all records in second table.

Syntax of Cross Join

```
SELECT columns FROM table1 CROSS JOIN table2 ON table1.column = table2.column;
```

Example of Cross Join

```
SELECT * FROM Movie CROSS JOIN Viewer ON
Movie.mid=Viewer.mid;
```

```
mysql> SELECT * FROM Movie CROSS JOIN Viewer ON Movie.mid=Viewer.mid;
+-----+--------------------+-----------+------+-----------+------+
| mid | title              | category  | id   | name      | mid  |
+-----+--------------------+-----------+------+-----------+------+
|   4 | Avengers           | Action    |  101 | Swapnil   |    4 |
|   1 | Real Steel(2012)   | Animation |  102 | Aakash    |    1 |
|   3 | Madras Cafe        | Action    |  103 | Prashant  |    3 |
+-----+--------------------+-----------+------+-----------+------+
```

MySQL View

A view is a database object that has no values. Its contents are based on the base table. It contains rows and columns similar to the real table.

In MySQL, the View is a virtual table created by a query by joining one or more tables. It is operated similarly to the base table but does not contain any data of its own.

The View and table have one main difference that the views are definitions built on top of other tables (or views). If any changes occur in the underlying table, the same changes reflected in the View also.

Create View

syntax to create a view:

```
CREATE VIEW view_name AS  SELECT columns FROM tables [WHERE conditions];
```

Syntax to display a view :

```
SELECT * FROM view_name;
```

Where,

view_name: It specifies the name of the VIEW that we want to create in MySQL.

Example of create view:

```
mysql> select * from Class1;
+------+--------+---------+-------+
| roll | name   | subject | marks |
+------+--------+---------+-------+
|    1 | Harsh  | Physics |    73 |
|    2 | Gita   | CS      |    79 |
|    3 | Lalit  | English |    85 |
|    4 | Rahul  | CS      |    85 |
+------+--------+---------+-------+
```

Let us understand it with the help of an example. Suppose our database has a table **class1**, and we are going to create a view based on this table. Thus, the below example will create a VIEW name "**Paper**" that creates a virtual table made by taking data from the table courses.

```
CREATE VIEW Paper AS  SELECT subject  FROM Class1;
```

Now let's display this view as,

```
select * from Paper;
```

```
mysql> select * from Paper;
+---------+
| subject |
+---------+
| Physics |
| CS      |
| English |
| CS      |
+---------+
```

MySQL Update VIEW

In MYSQL, the ALTER VIEW statement is used to modify or update the already created VIEW without dropping it.

syntax to update the existing view:

```
ALTER VIEW view_name AS  SELECT columns  FROM table [WHERE conditions];
```

Example To update a view:

```
ALTER VIEW Paper AS  SELECT name,subject  FROM Class1;
```

```
mysql> select * from Paper;
+--------+---------+
| name   | subject |
+--------+---------+
| Harsh  | Physics |
| Gita   | CS      |
| Lalit  | English |
| Rahul  | CS      |
+--------+---------+
```

MySQL Drop VIEW

We can drop the existing VIEW by using the DROP VIEW statement.

Syntax to drop a View:

```
DROP VIEW [IF EXISTS] view_name;
```

Example to drop a view:

```
DROP VIEW Paper;
```

```
mysql> DROP VIEW Paper;
Query OK, 0 rows affected (0.24 sec)

mysql> select * from Paper;
ERROR 1146 (42S02): Table 'june.paper' doesn't exist
```

Indexing

The INDEX is used to create and retrieve data from the database very quickly. An Index can be created by using a single or group of columns in a table.

When the index is created, it is assigned a ROWID for each row before it sorts out the data.

Types of Index:

1) Simple Index 2) Unique Index 3) Composite Index

1) Simple Index:

A Simple is an index that allows duplicate values in a table.

If we want to index the values in a column in a descending order, we can add the reserved word DESC after the column name.

Syntax To Create Simple Index :

```
CREATE INDEX indexname ON Tablename(columnname);
```

Syntax to Show index:

```
SHOW INDEXES FROM Tablename;
```

```
mysql> select * from Shirts;
+----+--------------+--------+
| id | name         | size   |
+----+--------------+--------+
|  1 | t-shirt      | medium |
|  2 | casual-shirt | small  |
|  3 | formal-shirt | large  |
+----+--------------+--------+
3 rows in set (0.00 sec)
```

Example: Consider the table Shirts on which we are going to apply Simple index Constraint

```
CREATE INDEX Demoindex ON Shirts(id);
```

```
SHOW INDEXES FROM Shirts;
```

2) Unique Index:

In MySQL we can add Index on one or more than one row.

A unique index means that two rows cannot have the same index value.

Syntax To Create Unique Index :

```
CREATE UNIQUE INDEX indexname ON Tablename(columnname);
```

Syntax to Show index:

```
SHOW INDEXES FROM Tablename;
```

To create a unique index, we just have to add UNIQUE statement before INDEX Statement present in Syntax of simple index.

Example: Consider the table Shirts on which we are going to apply Unique index Constraint

Example: Consider the table Shirts on which we are going to apply Unique index Constraint

```
CREATE UNIQUE INDEX Uniindex ON Shirts(id);
```

```
SHOW INDEXES FROM Shirts;
```

3) Composite Index:

A composite index is an index that is used on multiple columns. It is also known as a multiple column index.

MySQL allows the user to create a composite index which can consist of up to 16 columns.

Syntax To Create Unique Index :

```
CREATE INDEX indexname ON Tablename(column1,column2…);
```

Syntax to Show index:

```
SHOW INDEXES FROM Tablename;
```

Example: Consider the table Shirts on which we are going to apply Composite index Constraint

```
CREATE  INDEX compindex ON Shirts(id,name);
```

```
SHOW INDEXES FROM Shirts;
```

DROP INDEX: The DROP INDEX statement is used to delete an index in a table.

Syntax:

```
ALTER TABLE Tablename DROP INDEX Indexname;
```